

2D Sports Posture Analysis Pipeline

Prof. Francesco La Rosa
Mehdi Talebikhatir

Contents

Abstract	2
1. Project Objective	2
2. Pipeline Architecture.....	2
3. Experimental Setup.....	2
4. Execution Commands.....	3
5. Implemented Stages and Future Roadmap	4
6. Live Pose vs Offline Frame-Based Pipeline	4
7. Results and Visual Interpretation.....	5
7.1 Annotated Pose Frames.....	5
7.2 Biomechanical Feature Plots	6
8. Tool and Model Choice.....	8
9. Limitations and Future Work	9
10. Conclusion	9
References.....	9

Abstract

This report describes the current implementation of a modular computer-vision pipeline for sports posture analysis from video input. The implemented system performs video frame extraction, 2D human pose estimation, live pose visualization, temporal smoothing, pose-quality checking, and OpenCap-inspired 2D biomechanical surrogate feature extraction using OpenCV, RTMLib/RTMPose, ONNX Runtime, pandas, and Matplotlib [1], [2], [3], [4].

The system should be interpreted as a research prototype for 2D image-plane posture analysis, not as a full OpenCap system, not as calibrated 3D biomechanics, and not as a clinical or injury-risk diagnostic tool. This distinction is scientifically important because OpenCap and Pose2Sim-style workflows use stronger capture assumptions such as calibrated video, multi-view reconstruction, musculoskeletal modeling, or validated 3D kinematic pipelines [5], [6], [7].

1. Project Objective

The objective of this project is to build a reproducible and extensible pipeline that converts a sports or exercise video into interpretable 2D posture outputs. The current outputs include extracted frames, body keypoints, annotated skeleton frames, smoothed keypoint trajectories, pose-confidence summaries, knee and hip angle curves, trunk-lean estimates, and instantaneous left-right angle differences [1], [3], [8].

The project is designed as an academic prototype rather than a finished clinical system. Its value is that each processing stage is explicit, inspectable, and reproducible, which makes the pipeline suitable for demonstrations, coursework, internship reporting, and future validation against stronger biomechanical references [5], [6], [7].

2. Pipeline Architecture

The repository follows a layered architecture. Reusable scientific and engineering logic is placed under `src/hpa/`, while executable command-line scripts are placed under `src/scripts/`. This separation makes the project easier to test, extend, and reuse because model code, video I/O, smoothing, biomechanics, and scripts are not mixed into one monolithic file [1], [9].

```
Raw video
-> Video I/O: OpenCV metadata reading and frame extraction
-> Pose layer: RTMLib person detection and RTMPose 2D keypoints
-> Visualization: annotated frames and live OpenCV display
-> Smoothing: per-person, per-keypoint temporal moving average
-> Quality checking: confidence and low-confidence statistics
-> Biomechanics: OpenCap-inspired 2D surrogate features
-> Reporting: tables, plots, figures, limitations, and future work
```

This architecture is appropriate because each stage has a different failure mode. Video decoding can fail because of file or codec issues, pose estimation can fail because of occlusion or detection errors, smoothing can hide fast movement if the window is too large, and biomechanical interpretation can be misleading if 2D image-plane angles are treated as 3D anatomical measurements [2], [3], [7], [8].

3. Experimental Setup

The final experiment in this report used the treadmill video `data/raw/videos/tredmil0.mp4`. The video has 888 frames, 30 FPS, 29.60 seconds duration, and 640 x 360 resolution. Full-frame extraction was used with `step=1`, so every decoded frame was processed by the downstream pose-estimation and feature-extraction stages [2], [7].

The configured local model paths were `models/detection/yolox_m_humanart.onnx` for human detection and `models/pose/rtmpose_m_body7.onnx` for 2D body pose estimation. The pose pipeline used RTMLib with the ONNX Runtime backend, CUDA device execution, and lightweight mode for practical local throughput [3], [4], [10]. The confidence threshold used for quality reporting and OpenCap-style feature flagging was 0.3, and the temporal smoothing window was 5 frames. These parameters were selected as simple, readable prototype defaults: the

threshold catches very unreliable keypoints, while the short moving-average window reduces frame-to-frame jitter without intentionally changing the confidence values [3], [8], [7].

Experimental item	Value
Input video	data/raw/videos/tredmil0.mp4
Resolution	640 x 360
Duration	29.60 s
Frame rate	30 FPS
Total video frames	888
Frame extraction step	1
Saved frames	888
Annotated frames	888
Detector model path	models/detection/yolox_m_humanart.onnx
Pose model path	models/pose/rtmpose_m_body7.onnx
Backend / device	ONNX Runtime / CUDA
Mode	lightweight
Confidence threshold	0.3
Smoothing window	5 frames

4. Execution Commands

The following commands reproduce the current pipeline setup and execution sequence. They are included explicitly so that the experiment can be repeated and checked from a clean environment.

```
python -m venv .venv .venv\Scripts\activate pip install -r requirements.txt pip install -e . python
src/scripts/download_models.py python src/scripts/extract_frames.py --video data/raw/videos/tredmil0.mp4 --output
data/interim/frames --step 1
python src/scripts/run_pose_on_frames.py --input data/interim/frames --output-csv
data/processed/keypoints/keypoints.csv --output-frames data/outputs/annotated_frames --device cuda --mode
lightweight
python src/scripts/smooth_keypoints.py --input data/processed/keypoints/keypoints.csv --output
data/processed/smoothed_keypoints/smoothed_keypoints.csv --quality-report
data/outputs/reports/pose_quality.csv --window-size 5
python src/scripts/extract_opencap_style_features.py --input
data/processed/smoothed_keypoints/smoothed_keypoints.csv --output
data/processed/biomechanics/opencap_style_features.csv --plots-dir data/outputs/reports/opencap_style_plots --
confidence-threshold 0.3
python src/scripts/live_pose_demo.py --video data/raw/videos/tredmil0.mp4 --device cuda --mode lightweight
--max-width 640
```

The project also keeps simple default commands for quick local testing. The frame-extraction script still needs a video path because the input video should never be guessed silently, while the later scripts can use their configured default

input and output paths. These defaults are useful during development, but the full commands above are preferred for academic reproducibility because they make the input and output paths explicit.

```
python src/scripts/extract_frames.py --video data/raw/videos/tredmil10.mp4
python src/scripts/run_pose_on_frames.py python
src/scripts/smooth_keypoints.py python
src/scripts/extract_opencap_style_features.py
```

5. Implemented Stages and Future Roadmap

The current implementation covers the complete 2D prototype path from video to posture features. Future modules are already represented in the repository structure, but they should remain clearly labeled as future work until they are implemented, tested, and validated [1], [6], [5].

Pipeline component	Current status	Notes
Video input and frame extraction	Implemented	OpenCV metadata and frame extraction
Human detection	Implemented inside RTMLib stage	Detector outputs are used internally for pose inference
2D pose estimation	Implemented	RTMLib/RTMPose outputs COCO-style 2D keypoints
Live visualization	Implemented	OpenCV display, FPS overlay, optional output saving
Pose smoothing	Implemented	Moving average over x and y per person/keypoint
Pose quality checking	Implemented baseline	Aggregate confidence and low-confidence metrics
OpenCap-style 2D features	Implemented	2D surrogate angles, not full OpenCap biomechanics
Multi-person tracking	Future work	Needed for crowded sports scenes
3D pose estimation	Future work	Requires calibrated multi-view data or a dedicated 3D workflow
Action recognition	Future work	Requires labeled actions and temporal modeling
Fatigue/injury-risk scoring	Future work	Requires validation, domain labels, and clinical caution
Dashboard	Future work	Useful after metrics and validation are stable

6. Live Pose vs Offline Frame-Based Pipeline

The live pose demo is valuable for demonstrations because it gives immediate visual feedback, shows the system operating frame by frame, and allows a viewer to quickly understand how the skeleton overlay behaves on the input video. However, live execution is more sensitive to runtime speed, display latency, GPU setup, and frame resizing choices [2], [3], [10].

The offline frame-based pipeline is better for reproducible experiments because it saves intermediate frames, keypoint CSV files, annotated images, quality reports, and biomechanical plots. This makes errors easier to inspect and makes the results easier to include in an academic report or presentation [1], [2], [8].

Criterion	Live pose demo	Offline frame-based pipeline
Main purpose	Real-time demonstration	Reproducible analysis
Speed priority	High	Medium
Debugging	Visual only unless saved	Strong, because intermediate files exist
Reproducibility	Lower unless CSV/video are saved	Higher because outputs are persistent
Best use	Demo and quick inspection	Reported experiment and feature extraction
Accuracy interpretation	Useful for visual plausibility	Better for quantitative summaries and plots

7. Results and Visual Interpretation

The pose-estimation output contains 15,096 keypoint rows, which equals 888 frames multiplied by 17 COCO-style body keypoints for one detected person. This confirms that the clean single-person treadmill video produced a temporally consistent pose output without multi-person identity mixing in this experiment [3], [11].

The average keypoint confidence was 0.843688, and the low-confidence count below the 0.3 threshold was zero. This indicates that the selected video is suitable for demonstrating the prototype pipeline, although confidence scores alone do not prove anatomical correctness and must be paired with visual checks of annotated frames [3], [7].

Quality metric	Value
Processed frames	888
Keypoint rows	15,096
Unique detected persons	1
Keypoints per frame/person	17
Average confidence	0.843688
Low-confidence count below 0.3	0
Low-confidence ratio below 0.3	0.0000
Smoothed keypoint rows	15,096
Biomechanical feature rows	888

The OpenCap-inspired feature output contains one feature row per processed frame/person. The extracted knee angles show strong cyclic variation, the hip angles show a more bounded cyclic pattern, and the trunk-lean angle remains close to zero degrees, which is visually consistent with an upright treadmill-running posture [5], [7].

Feature	Mean	Std	Min	Median	Max
Left knee angle	121.962	34.419	64.077	131.240	171.589
Right knee angle	139.678	26.516	67.815	151.767	170.573
Left hip angle	165.653	9.381	144.927	167.220	179.665
Right hip angle	167.097	9.846	144.159	168.948	179.981
Trunk lean angle	1.214	0.752	0.000	1.176	2.769
Knee angle asymmetry	49.033	27.972	0.064	51.134	91.522
Hip angle asymmetry	15.008	9.020	0.079	12.950	34.542
Mean pose confidence	0.844	0.024	0.778	0.843	0.898

The large same-frame knee angle difference should be interpreted as instantaneous left-right angular difference, not clinical asymmetry. During running, one leg can be flexed while the other is extended, so a high same-frame left-right difference can reflect gait phase rather than an error or injury-related asymmetry [6], [7].

7.1 Annotated Pose Frames

The annotated frames provide a qualitative check that the estimated skeleton is attached to the visible body. Frame 000014 shows the detected runner with visible shoulder, hip, knee, and ankle landmarks, and the skeleton overlay is suitable for prototype-level visual interpretation [2], [3].



Annotated pose frame 000014

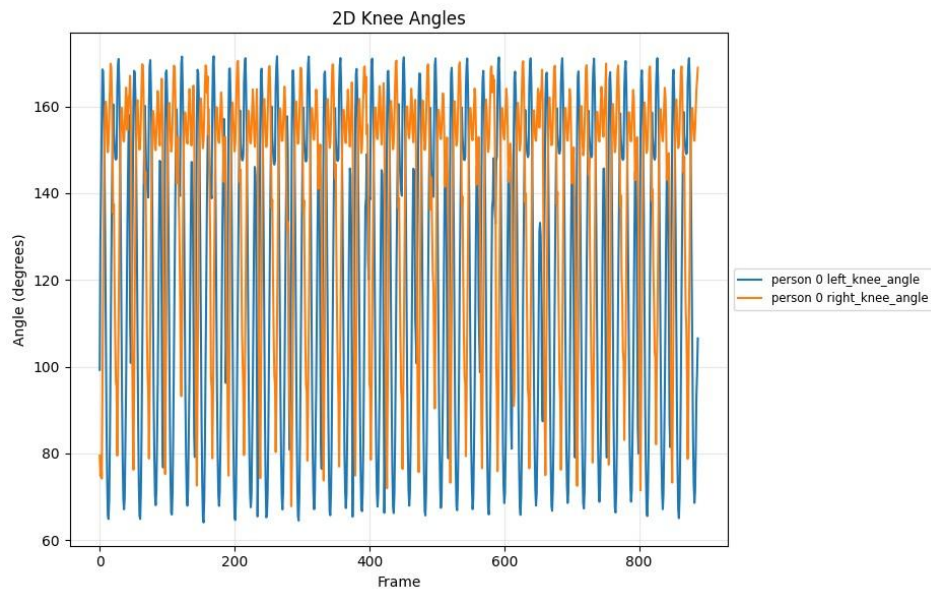
Frame 000375 shows a different gait phase, where one leg is flexed and the other is more extended. This visual evidence supports the interpretation that high instantaneous knee-angle differences can be expected in running sequences [6], [7].



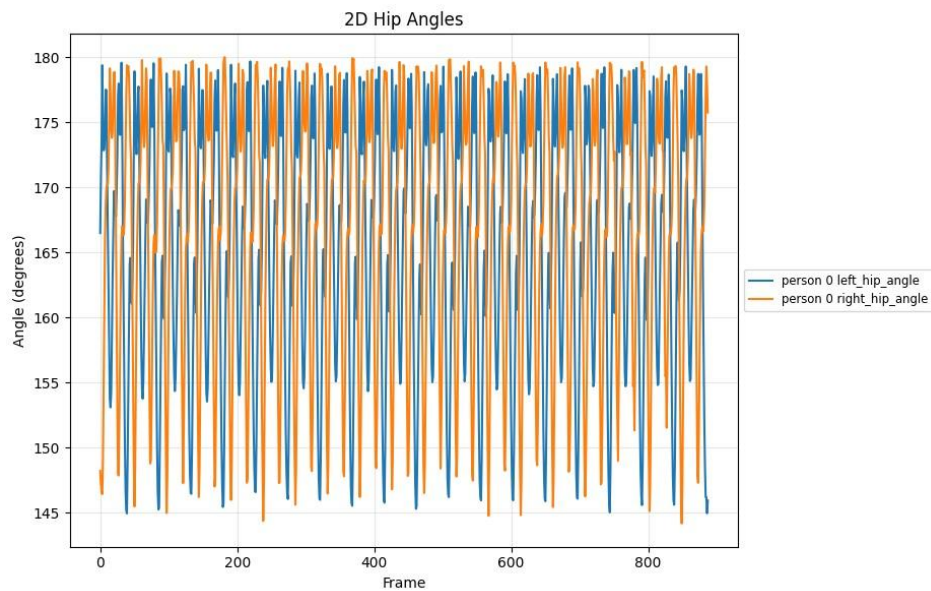
Annotated pose frame 000375

7.2 Biomechanical Feature Plots

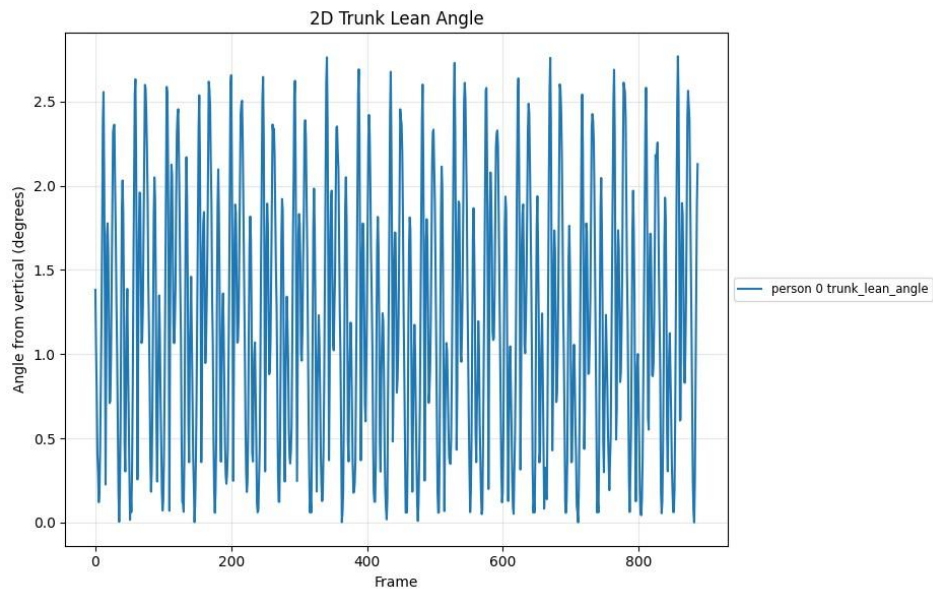
The knee-angle plot shows repeated oscillations across the treadmill sequence. This is consistent with cyclic lower-limb motion, but the values remain 2D image-plane estimates and should not be reported as validated 3D knee joint angles [5], [7].

*Knee angle visualization*

The hip-angle plot is more bounded than the knee-angle plot, which is expected because the hip and trunk region usually show less apparent image-plane variation than the distal lower limb during treadmill running [6], [7].

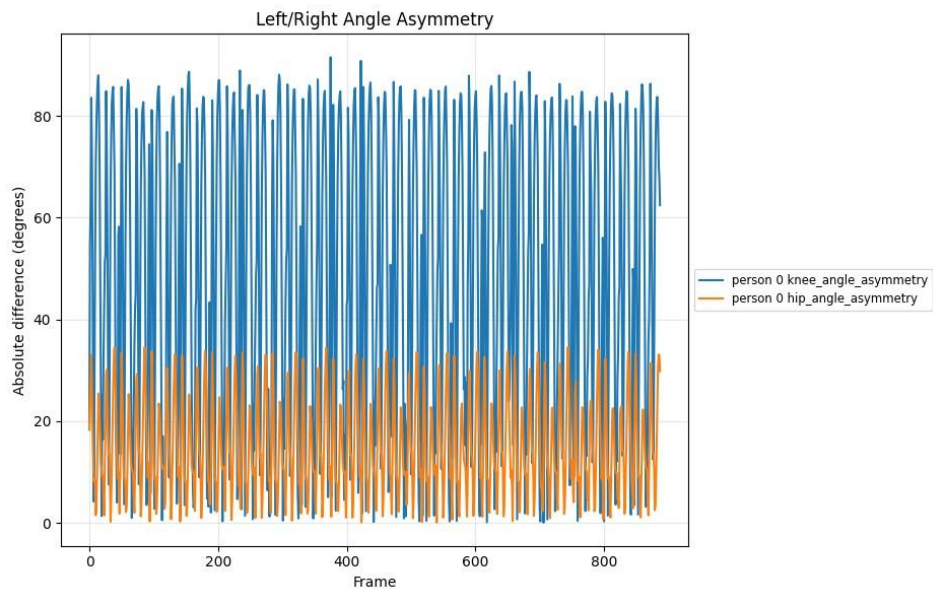
*Hip angle visualization*

The trunk-lean plot remains close to zero degrees throughout the sequence. This matches the visual impression of an upright treadmill-running posture and supports internal consistency between the annotated frames and the extracted 2D trunk feature [5], [7].



Trunk lean visualization

The left-right angle-difference plot should be used as a screening and visualization feature only. The current system does not yet segment gait cycles or compare left and right limbs at matched movement phases, so this metric is not equivalent to clinical gait asymmetry [6], [7].



Instantaneous angle-difference visualization

8. Tool and Model Choice

RTMLib and RTMPose were selected because they provide a practical balance between installation simplicity, speed, ONNX deployment, and modern 2D pose-estimation quality. This is appropriate for a student research prototype because it avoids the heavier setup of full MMPose while still using models from the same broader ecosystem [3], [4], [12].

ONNX Runtime was selected because it provides a standard deployment path for ONNX models and supports CUDA acceleration when the CUDA provider and dependencies are configured correctly. This makes the runtime configuration important to report because CPU and CUDA execution can produce very different throughput in live and batch pose-estimation workflows [10].

Option	Strength	Limitation	Role in this project
--------	----------	------------	----------------------

RTMLib + RTMPose	Simple API, ONNX-friendly, fast practical deployment	Less flexible than full MMPose	Current main implementation
Full MMPose	Large model zoo and research flexibility	Heavier setup and configuration	Future benchmarking option
MediaPipe Pose	Very easy live deployment	Less aligned with RTMPose/Pose2Sim workflow	Possible live baseline
VITPose	Strong transformer-based pose-estimation baseline	More demanding deployment	Future accuracy comparison
Pose2Sim	3D markerless sports kinematics workflow	Requires calibrated multi-view data	Future 3D extension
OpenCap	Validated smartphone-video movement dynamics platform	Not the same as this 2D prototype	Scientific inspiration and comparison target

9. Limitations and Future Work

The main limitation is that the current pipeline estimates 2D image-plane keypoints and 2D image-plane angles. Camera viewpoint, subject rotation, out-of-plane motion, occlusion, and lens distortion can change apparent 2D angles even when true 3D anatomical joint angles are different [7].

The second limitation is that human detection is currently used inside the RTMLib pose-estimation stage, but the project does not yet export independent detection files or implement robust multi-person tracking. This is acceptable for the clean single-person treadmill experiment, but it is not sufficient for crowded sports scenes where identity switches and occlusions are common [4], [13].

The third limitation is that the current smoothing approach is a simple moving average. This is clear and reproducible, but it can blur fast movements if the window is too large, so the smoothing window must always be reported as an experimental parameter [8], [7].

The most important future work is validation. The project should eventually compare its keypoints or angles against manually annotated frames, an external benchmark, OpenCap output, Pose2Sim output, or calibrated motion-capture data before making stronger biomechanical claims [5], [6], [7].

10. Conclusion

The current repository implements a coherent and reproducible 2D computer-vision pipeline for sports posture analysis. It successfully runs from raw video to extracted frames, RTMPose keypoints, annotated skeleton frames, smoothed trajectories, quality reports, and OpenCap-inspired 2D posture features [1], [2], [3].

The current results are suitable for a prototype/demo stage because they show complete data flow, readable outputs, and interpretable movement plots on a clean treadmill-running video. The report and presentation should continue to state clearly that the system produces 2D surrogate posture features only and that full OpenCap-style or Pose2Sim-style biomechanical validation remains future work [5], [6], [7].

References

- [1] M. Talebi, HCI-for-Human-Posture-Analysis, GitHub repository. <https://github.com/mehditalebi01/HCI-for-Human-Posture-Analysis>
- [2] OpenCV, Getting Started with Videos. https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html
- [3] T. Jiang et al., RTMPose: Real-Time Multi-Person Pose Estimation based on MMPose, arXiv:2303.07399, 2023. <https://arxiv.org/abs/2303.07399>
- [4] T. Jiang, RTMLib, GitHub repository. <https://github.com/Tau-J/rtmlib>
- [5] S. D. Uhrich et al., OpenCap: Human movement dynamics from smartphone videos, PLOS Computational Biology, 2023. <https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1011462>
- [6] D. Pagnon, M. Domalain, and L. Reveret, Pose2Sim: An End-to-End Workflow for 3D Markerless Sports Kinematics, Sensors, 2021. <https://www.mdpi.com/1424-8220/21/19/6530>
- [7] Sensors, 2D/3D joint-angle interpretation reference for video-based kinematics limitations. <https://www.mdpi.com/1424-8220/22/5/1729>
- [8] pandas, DataFrame.rolling documentation.

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.rolling.html> [9] Python Packaging User Guide, src layout vs flat layout. <https://packaging.python.org/en/latest/discussions/src-layout-vs-flat-layout/> [10] ONNX Runtime, CUDA Execution Provider. <https://onnxruntime.ai/docs/execution-providers/CUDA-ExecutionProvider.html> [11] T.-Y. Lin et al., Microsoft COCO: Common Objects in Context, arXiv:1405.0312, 2014. <https://arxiv.org/abs/1405.0312> [12] OpenMMLab, MMPose Documentation. <https://mmpose.readthedocs.io/> [13] Z. Ge et al., YOLOX: Exceeding YOLO Series in 2021, arXiv:2107.08430, 2021. <https://arxiv.org/abs/2107.08430> [14] Google AI Edge, MediaPipe Pose Landmarker. https://developers.google.com/mediapipe/solutions/vision/pose_landmarker [15] Y. Xu et al., ViTPose: Simple Vision Transformer Baselines for Human Pose Estimation, arXiv:2204.12484, 2022. <https://arxiv.org/abs/2204.12484>